HANDS-ON EXERCISE

Insurance Ontology Baseline

Week 1 · Day 3 · Self-Paced Lab, Fully Self-Contained · Prepared by Rajesh Pasham

Before You Begin

This is a self-paced lab, and it is fully self-contained: everything you need — including your reference data — is provided with this document. You do not need any file from the trainer beyond what's already attached, and you do not need a second account or a shared Project. Every part of this lab, including the security test in Part 8, is designed to be completed entirely on your own individual Palantir Developer Tier account.

How Long This Takes

Budget 3–4 hours of focused work. This is longer than a typical Day 3 lab because it covers the client's full "Insurance Ontology Baseline" specification in one session, at production-realistic data volume. If you don't finish everything today, that's expected — see "If You Run Out of Time" at the end of this document.

You Are Working Entirely Solo

Every part of this lab, including the access-control test in Part 8, is designed around a single individual Developer Tier account with no shared Project, no batch-mate, and no second user. Where the underlying platform concept would normally involve another person, this lab shows you how to validate the same mechanism using only your own account.

What You Need Before Starting

- Your individual Palantir Developer Tier account, already signed in
- A Project of your own to work in — create one now if you haven't already
- The four attached reference CSV files (policyholders.csv, policies.csv, claims.csv, payments.csv) — 300 rows each — or the generation script in Part 1 if you'd rather produce them yourself
- Today's Day 1 and Day 2 study guides open for reference — this lab assumes that material, it doesn't repeat it

Getting Unstuck Without the Trainer

- Re-read the relevant step first — most issues are a missed sub-step, not a platform problem
- Check the Troubleshooting appendix at the end of this document for the specific symptom you're seeing
- Search the exact error message in the Palantir Developer Community (community.palantir.com) — most Developer Tier issues have already been asked about
- Note the blocker in your validation checklist and bring it to Day 4 — a documented blocker is a legitimate outcome of a self-paced lab

Lab Outcome

By the end of this lab, you will have: a secured, agent-ready Insurance Ontology (seven Object Types with governed relationships) and a production-oriented, incremental pipeline baseline, built at a realistic data volume — exactly as specified in the curriculum, entirely within your own individual account.

Lab Roadmap

Part	Task
1	Load your reference policyholder, policy, claim, and payment data (300 rows each)
2	Build an incremental claims-transformation pipeline
3	Configure schema and data-quality checks
4	Implement branching and controlled promotion
5	Create the seven Ontology objects
6	Define complex object relationships
7	Apply RBAC and ABAC controls
8	Test restricted access to high-value and sensitive claims — solo method
9	Review lineage and downstream dependencies

Part 1 — Load Your Reference Data

The curriculum assumes “preconfigured policy, customer, claim and payment data.” At 300 rows per dataset, that's realistic enough to behave like production data — filters, aggregations, and security policies will all touch a meaningful number of rows, not one or two examples. You do not need to type or paste this data by hand.

Dataset Summary

All four files use a fixed random seed, so this exact data is what both the attached files and Method B's script below produce — every number in this document is accurate for your dataset.

Dataset	Rows	Notable Characteristics
policyholders.csv	300	Unique policyholderId PH001–PH300
policies.csv	300	Unique policyId POL001–POL300; each references a valid policyholderId
claims.csv	300	Exactly 6 rows have an empty policyId (data-quality test target). 34 rows have claimAmount above 25,000 (high-value security test target)
payments.csv	300	Every claimId references an Approved claim; 101 claims are Approved, so some claims have more than one payment. 63 payments have paymentStatus of Pending

Method A — Use the Attached Files

1. Download the four attached CSV files: policyholders.csv, policies.csv, claims.csv, payments.csv.
2. Inside your Project, select + New, then choose Upload files (or drag all four directly into the file browser).
3. Rename each dataset clearly — for example, raw_policyholders — using the rename option in the dataset's menu.

Method B — Generate the Same Data Yourself

Use this method if you'd rather not handle files at all, or want the more production-realistic pattern of generating reference data as code. Because it uses the same random seed, it produces byte-for-byte the same data as the attached files.

4. Inside your Project, select + New, then choose Code Repository. Choose Python.
5. Create a file with the generation logic below. This version writes local CSVs for illustration — to write directly to Foundry datasets, wrap each dataset's list in an @transform_df function with an Output("/YourProject/raw_x") and return ctx.spark_session.createDataFrame(pd.DataFrame(rows)).

```
import random, csv
from datetime import date, timedelta

random.seed(42)  # same seed as the attached files — your output will match exactly

FIRST_NAMES = ["Ananya", "David", "Fatima", "James", "Meera", "Lucas", "Priya", "Thomas", ...]
LAST_NAMES = ["Rao", "Chen", "Al-Sayed", "O'Brien", "Iyer", "Silva", "Nair", "Weber", ...]

def rand_date(y1, y2):
    start = date(y1, 1, 1)
    return start + timedelta(days=random.randint(0, (date(y2, 12, 31) - start).days))

# Policyholders (300)
policyholders = []
for i in range(1, 301):
    fn, ln = random.choice(FIRST_NAMES), random.choice(LAST_NAMES)
    policyholders.append({
        "policyholderId": f"PH{i:03d}", "name": f"{fn} {ln}",
        "dateOfBirth": rand_date(1955, 2002).isoformat(),
        "contactEmail": f"{fn.lower()}.{ln.lower()}{i}@example.com",
    })

# Policies (300) — policyholderId chosen with replacement, so counts vary per holder
LIMITS = [25000, 30000, 40000, 50000, 60000, 75000, 100000, 150000]
policies = []
for i in range(1, 301):
    ph = random.choice(policyholders)
    start = rand_date(2024, 2025)
    policies.append({
        "policyId": f"POL{i:03d}", "policyholderId": ph["policyholderId"],
        "policyLimit": random.choice(LIMITS), "coverageType": random.choice(["Auto", "Home"]),
        "startDate": start.isoformat(), "endDate": (start + timedelta(days=365)).isoformat(),
    })

# Claims (300) — exactly 6 rows get an empty policyId; ~15% of the rest exceed 25,000
null_policy_rows = set(random.sample(range(1, 301), 6))
claims = []
for i in range(1, 301):
    policy_id = "" if i in null_policy_rows else random.choice(policies)["policyId"]
    high_value = random.random() < 0.15
    amount = random.randint(25001, 60000) if high_value else random.randint(500, 25000)
    status = random.choice(["Approved", "UnderReview", "Rejected"])
    claims.append({
        "claimId": f"CLM{i:03d}", "policyId": policy_id, "claimAmount": amount, "status": status,
        "fraudScore": round(random.uniform(0.01, 0.95), 2),
        "submissionDate": rand_date(2025, 2025).isoformat(),
    })

# Payments (300) — drawn from Approved claims only; ~20% Pending
approved = [c["claimId"] for c in claims if c["status"] == "Approved"]
payments = []
for i in range(1, 301):
    claim_id = random.choice(approved)
    amt = next(c["claimAmount"] for c in claims if c["claimId"] == claim_id)
    payments.append({
        "paymentId": f"PAY{i:03d}", "claimId": claim_id, "amount": amt,
        "paymentDate": rand_date(2025, 2025).isoformat(),
        "paymentStatus": random.choice(["Completed"] * 8 + ["Pending"] * 2),
    })
```

```
# Write CSVs — or in Foundry, wrap each list in @transform_df + pd.DataFrame as in the earlier
example
for name, rows, fields in [
    ("policyholders", policyholders, ["policyholderId","name","dateOfBirth","contactEmail"]),
    ("policies", policies,
["policyId","policyholderId","policyLimit","coverageType","startDate","endDate"]),
    ("claims", claims,
["claimId","policyId","claimAmount","status","fraudScore","submissionDate"]),
    ("payments", payments, ["paymentId","claimId","amount","paymentDate","paymentStatus"]),
]:
    with open(f"{name}.csv", "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fields)
        w.writeheader(); w.writerows(rows)
```

6. Run it (or adapt it into four @transform_df functions and build them) to produce your four datasets.

Preview: First Rows of Each Dataset

For reference, here's what the first few rows of each file actually look like, so you can sanity-check your upload or generation.

File	First Data Row
policyholders.csv	PH001, Thomas Rao, 1979-09-04, thomas.rao1@example.com
policies.csv	POL001, PH241, 25000, Auto, 2024-10-25, 2025-10-25
claims.csv	CLM001, POL032, 17496, Approved, 0.19, 2025-12-06
payments.csv	PAY001, CLM192, 4368, 2025-09-24, Pending

Applying and Reviewing the Schema (Method A Only)

If you used Method A, a freshly uploaded CSV has no schema yet — Foundry treats it as unstructured text until you apply one. Skip this section if you used Method B.

1. Open the dataset. Select Apply a schema. Foundry infers column names and types from a sample of the file.
2. Select Edit schema to review the inferred types. Correct any column that was inferred incorrectly — dates and numeric IDs are the most common mistakes.
3. Confirm the row count is 300 for all four datasets.

> **Checkpoint — Confirm Before Continuing**
>
> All four datasets — raw_policyholders, raw_policies, raw_claims, raw_payments — exist in your Project with 300 rows each, regardless of which method you used.

Part 2 — Build an Incremental Claims-Transformation Pipeline

You'll build this in Pipeline Builder, the no-code pipeline tool. This directly applies the incremental-processing concept from Day 1.

2.1 Create the Pipeline

4. Inside your Project, select + New, then choose Pipeline Builder.
5. Name the pipeline claims_transformation_pipeline.
6. In the empty graph area, select Add Foundry data and choose raw_claims as your input.

2.2 Mark the Input as Incremental

7. Select the raw_claims input node. Below the dataset, mark it as Incremental.

8. Confirm a blue badge appears in the top-right corner of the node, indicating the change took effect.

From this point, any transform connected downstream of this input will also show the incremental indicator — this is how you can visually confirm incremental behavior is propagating through your pipeline.

2.3 Add a Transform

9. Add a transform node downstream of raw_claims.
10. Configure it to filter out any row where claimId or policyId is empty — this removes the 6 claims with an empty policyId, and feeds directly into Part 3's data-quality work.
11. Add a derived column, claimAgeDays, calculated from the claim's submission date to today's date — you'll use this later when reasoning about claim prioritization.

2.4 Add the Output Dataset

12. Select Add next to the Dataset output type in the outputs panel.
13. Rename the output to claims_transformed.
14. Use Use updated schema to pull the schema from your transform node automatically.
15. Leave the write mode on Default — this is the recommended setting for incremental batch pipelines like this one.

2.5 Deploy

16. Select Deploy. Wait for the first build to complete — this first run processes the full dataset (a snapshot), which is expected and normal.
17. Confirm claims_transformed now exists with 294 rows (300 source claims minus the 6 with an empty policyId).

> **Checkpoint — Confirm Before Continuing**
> claims_transformed exists with 294 rows, and its input (raw_claims) shows the incremental badge.

Part 3 — Configure Schema and Data-Quality Checks

This Part adds guardrails so that bad data fails the pipeline build loudly, instead of flowing silently into your Ontology.

3.1 Add an Expectation on Your Output

18. Open claims_transformed inside Pipeline Builder. Select the output node.
19. Add an expectation requiring claimAmount to be greater than zero.
20. Add a second expectation requiring policyId to be non-null (this reinforces the filter from Part 2, as a safety net rather than a duplicate).
21. Save your pipeline configuration.

3.2 Test That the Check Actually Works

An untested data-quality check is not a data-quality check — confirm it actually fails when it should.

22. Temporarily edit your transform to allow the 6 empty-policyId rows through, then re-deploy.
23. Confirm the build fails, and that the failure message references your policyId expectation.
24. Revert your transform back to the correct filter logic, then re-deploy and confirm the build succeeds with 294 rows again.

> **Checkpoint — Confirm Before Continuing**

> You have seen the pipeline fail on the empty-policyId rows, and succeed once the filter was restored. If you only ever saw it succeed, your expectation isn't actually being tested.

Part 4 — Implement Branching and Controlled Promotion

This is the branching workflow from Day 1, applied for real to the pipeline you just built.

4.1 Create a Feature Branch

25. From the top bar of your Project, create a new branch. Name it add-payment-status-flag.
26. Confirm you are now working on this branch, not on master — the branch indicator in the top bar should show your branch name.

4.2 Make a Change on the Branch

27. On this branch, add a new transform to claims_transformation_pipeline that joins in a hasOpenPayment flag from raw_payments — true if the claim has at least one payment with paymentStatus of Pending, false otherwise. About 63 of your 300 payments are Pending, so expect a meaningful share of claims to come back true.
28. Deploy this change. Confirm it builds successfully on your branch, without affecting the version on master.

4.3 Review, Then Merge

29. Review your own change as if you were a second reviewer: does the new column do what its name says?
30. Merge your branch back to master.
31. Confirm claims_transformed on master now includes the hasOpenPayment column, with a mix of true and false values.

> **Checkpoint — Confirm Before Continuing**
> Your pipeline change went through a branch, not a direct edit to master, and you can see the new column on master only after merging.

Part 5 — Create the Seven Ontology Objects

This is the core workflow from Day 1's Lab Reference Guide, repeated seven times. Work through them in the order below — later objects depend on earlier ones existing.

5.1 The Workflow, Reminder

Step	Action
1	Ontology Manager → New → Create object type
2	Select the backing dataset (or generate one if none exists yet)
3	Configure a primary key and a title key
4	Configure properties (adding any not already in the backing dataset)
5	Save

5.2 Object Specifications

Create these seven Object Types, in this order. Use the properties listed as a minimum — add more if you think the model needs them, and note why.

Object Type	Primary Key	Key Properties & Backing Data
Policyholder	policyholderId	name, dateOfBirth, contactEmail (PII). Backed by raw_policyholders (300 rows).
Insurance Policy	policyId	policyLimit, coverageType, startDate, endDate. Backed by raw_policies (300 rows).
Claim	claimId	claimAmount, status, fraudScore (sensitive), claimAgeDays, hasOpenPayment. Backed by claims_transformed from Part 2–4 (294 rows).
Claim Document	documentId	documentType, uploadDate, fileReference. Generate this Object Type without a pre-existing dataset — see 5.3 below.
Payment	paymentId	amount (sensitive), paymentDate, paymentStatus. Backed by raw_payments (300 rows).
Adjuster	adjusterId	name, team, caseloadCount. Generate this Object Type — see 5.3 below.
Investigation	investigationId	openedDate, status, riskLevel. Generate this Object Type — see 5.3 below.

5.3 Objects Without a Pre-Loaded Dataset

Claim Document, Adjuster, and Investigation don't have data behind them yet. When you reach Step 2 of the workflow for these three, choose the option to generate a dataset instead of selecting an existing one — Foundry will create an empty backing dataset matching the properties you define, which you can populate with a handful of sample rows directly afterward (five to ten each is enough at this data volume) so your Link Types in Part 6 have real data to connect.

> **Checkpoint — Confirm Before Continuing**
> All seven Object Types are visible in Ontology Manager, each with a valid primary key, and each showing real object instances when you preview them — hundreds for the dataset-backed ones, a handful for the generated ones.

Part 6 — Define Complex Object Relationships

Use the Day 1 guidance on choosing a relationship type: object type foreign keys for one-to-one or many-to-one cardinality, object-backed link types for many-to-many or when the relationship itself needs metadata.

Link	Cardinality	Relationship Type to Use
Policyholder → Insurance Policy	One-to-many	Object type foreign keys
Insurance Policy → Claim	One-to-many	Object type foreign keys
Claim → Claim Document	One-to-many	Object type foreign keys
Claim → Payment	One-to-many	Object type foreign keys
Claim → Investigation	One-to-one (not every claim has one)	Object type foreign keys, with the link optional
Investigation → Adjuster	Many-to-one	Object type foreign keys

Link	Cardinality	Relationship Type to Use
Claim → Adjuster (assigned adjuster)	Many-to-one	Object type foreign keys

Creating Each Link Type

32. From either Object Type's Overview page, select Create new link type within the link graph.
33. Choose Object type foreign keys for every link in the table above.
34. Select the foreign key property on the "many" side and the primary key it points to on the "one" side.
35. Save, then confirm the link appears on both connected Object Types' Overview pages.

Validate Full Navigability

36. In the Claim Object Type, filter for any claim where claimAmount is greater than 25,000 — pick one as your test case for the rest of this lab. Open it and confirm you can navigate to its Insurance Policy, then from there to its Policyholder — two hops, both working.
37. From the same Claim, confirm you can navigate to its Payments and Claim Documents, and (for the sample rows you added) an Investigation and Adjuster.

> **Checkpoint — Confirm Before Continuing**
> You can navigate Policyholder → Policy → Claim → Payment / Claim Document / Investigation → Adjuster entirely through the Ontology, without looking at any raw dataset.

Part 7 — Apply RBAC and ABAC Controls

This directly applies Day 2's RBAC and ABAC content to the Ontology you just built.

7.1 RBAC — Confirm Your Role

38. Open your Project's sharing settings. Confirm your own Role (you should be Owner, since you created this Project and everything in it).
39. You won't be able to test RBAC across multiple people in an individual account — instead, write down what Role you would assign to each Day 2 persona (Adjuster, Auditor, New Hire, Fraud Analyst) if this were a shared Project. This is a design exercise: bring it to Day 4 or Day 5 for review.

Persona	Role You'd Assign, and Why
Adjuster	
Auditor	
New Hire	
Fraud Analyst	

7.2 ABAC — Create a Marking

40. Navigate to Foundry Settings, then the Markings section.
41. Create a new Marking named High-Value Claim.

Important: creating a Marking automatically grants you Manage permissions on it — the ability to administer who is eligible. It does not automatically make you eligible yourself. That distinction is exactly what Part 8 tests.

7.3 Apply Object-Level Security to Claim

42. Open the Claim Object Type in Ontology Manager.

43. Configure an object security policy: objects where claimAmount is greater than 25,000 require the High-Value Claim Marking to view. This applies to the 34 high-value claims in your dataset.
44. Save. This is row-level security — it controls whether the Claim object is visible at all.

7.4 Apply Property-Level Security

45. On the same Claim Object Type, configure a property security policy on fraudScore, requiring the same High-Value Claim Marking.
46. Confirm you cannot select the object's primary key (claimId) when configuring this — property security policies apply to any property except the primary key, by design.

> **Checkpoint — Confirm Before Continuing**
>
> Claim has both an object security policy (row-level, based on claimAmount) and a property security policy (column-level, on fraudScore), both referencing the High-Value Claim Marking. You have Manage permissions on the Marking, but have not yet added yourself as eligible.

Part 8 — Test Restricted Access, Solo Method

This Part does not require a second account. It uses the distinction from Part 7.2 — managing a Marking is not the same as being eligible for it — to run a genuine before/after access test using only your own identity, against the test claim you picked in Part 6.

8.1 Before You Are Eligible

47. Without adding yourself to the High-Value Claim Marking's eligible members, attempt to view your test claim from Part 6 (one of the 34 with claimAmount above 25,000).
48. Confirm you cannot see the object, or cannot see it at all in a list view of Claim — this is object-level security working as configured, even though you administer the Marking.
49. If your environment has a Check access or similar simulation panel, use it to inspect your own effective access to this claim, and confirm it names the High-Value Claim Marking as the blocking factor.

8.2 After You Are Eligible

50. From Foundry Settings → Markings → High-Value Claim, add yourself to the Marking's eligible members (this is a separate action from the Manage permissions you already have).
51. Refresh and re-attempt to view your test claim. Confirm you can now see it, including its fraudScore value.
52. Pick any claim with claimAmount well under 25,000 and confirm you could see this one throughout, with or without the Marking — object security only applies above the threshold.

Record Your Results

Test	Observed Result
Your high-value test claim, before adding yourself as eligible	
Check access panel result, before eligibility (if available)	
Your high-value test claim, after adding yourself as eligible	
A low-value claim, for comparison	

> **Checkpoint — Confirm Before Continuing**
>
> You have direct evidence — not just configuration — that access was denied before you were eligible for the Marking and allowed after,

Test	Observed Result
using only your own account. If your test claim was visible even before Step 8.2, revisit 7.3–7.4 before continuing.	

Part 9 — Review Lineage and Downstream Dependencies

53. Open Data Lineage on your claims_transformed dataset.
54. Trace backward: confirm you can see raw_claims as its source, and (from Part 4) raw_payments feeding the hasOpenPayment column.
55. Trace forward: confirm you can see the Claim Object Type listed as a downstream consumer of claims_transformed.
56. Write one sentence on what would break, and for whom, if raw_claims changed its schema without warning.

> **Checkpoint — Confirm Before Continuing**
>
> You can explain, using the lineage graph rather than memory, exactly what feeds your Ontology and what depends on it.

Full Validation Checklist

- ☐ All four reference datasets exist with 300 rows each, loaded via Method A or Method B
- ☐ claims_transformation_pipeline runs incrementally, with a passing data-quality expectation
- ☐ You have seen the expectation fail on the empty-policyId rows and pass once corrected
- ☐ A pipeline change was made on a branch and merged, not edited directly on master
- ☐ All seven Object Types exist with valid primary keys and real instances
- ☐ All seven relationships are navigable end to end through the Ontology
- ☐ A High-Value Claim Marking exists, with object- and property-level security policies on Claim
- ☐ Restricted access was tested solo, before and after adding yourself as an eligible member
- ☐ Lineage for claims_transformed has been traced backward and forward

Reflection Questions

57. Part 7.2 pointed out that managing a Marking and being eligible for it are different things. Where else in an enterprise setting would that distinction matter — who should administer access without necessarily having it themselves?
58. If you had skipped Part 3's data-quality check, where downstream would a claim's missing policyId have surfaced instead — and would it have been obvious what caused it?
59. Now that fraudScore is protected by a Marking, what would you need to confirm before letting a Week 2 Copilot read the Claim object type?

If You Run Out of Time

Complete Parts 1–6 (the Ontology and pipeline itself) as the minimum before Day 4 — Parts 7–9 (security and lineage) can be finished as the first thing you do on Day 4 if needed, since Day 4's mentoring session will review your governance configuration anyway. Note in your validation checklist exactly which Parts remain, so Day 4's session can pick up precisely where you left off.

Troubleshooting

Symptom	Likely Cause & Fix
CSV upload succeeds but every column shows as text	You haven't applied a schema yet — select Apply a schema, then Edit schema to correct any misread types
Pipeline build fails immediately with no clear error	Check that every input node is connected to a transform, and every transform is connected to an output — an orphaned node is the most common cause
Incremental badge doesn't appear on downstream nodes	Confirm the upstream input is actually marked Incremental, not just added to the graph — this is a separate toggle from adding the data
Expectation doesn't fail even with the empty-policyId rows included	Confirm the expectation is attached to the output node, not a transform node, and that you redeployed after adding it
Can't select a primary key when creating an Object Type	The backing dataset likely has duplicate values in your intended key column — check for duplicates upstream before retrying
Link Type won't save	Cardinality mismatch — re-check whether the relationship is really one-to-many both ways, or if one side needs to be optional
Property security policy option is missing	Confirm you've already created and saved an object security policy on that Object Type first — property policies are configured on top of an existing object policy
You can see your test claim even before Part 8.2	You may have added yourself as eligible while creating the Marking in 7.2 — check the Marking's member list and remove yourself, then redo the before/after test correctly
Method B script runs but row counts don't match	Confirm random.seed(42) is set before any random calls, and that you haven't reordered the generation blocks — the sequence of random calls affects the output
Lineage graph looks empty or incomplete	Confirm you're viewing lineage on the dataset itself (claims_transformed), not on the Object Type — try both views if one seems incomplete